applied sciences

MDPI

Article

Design of a Security Framework for Multi-Agent Systems Based on Model Context Protocol in SOC Environments

Rodrigo Tavares de Pina Simões *, Xavier Larriva-Novo, Carmen Sánchez-Zas, Victor A. Villagrá * and Andrés I. Marín López

Departamento de Ingeniería de Sistemas Telemáticos, ETSI Telecomunicación, Universidad Politécnica de Madrid, 28040 Madrid, Spain; xavier.larriva.novo@upm.es (X.L.-N.); carmen.szas@upm.es (C.S.-Z.); andres.mlopez@upm.es (A.I.M.L.)

* Correspondence: rod.tpsim@gmail.com (R.T.d.P.S.); victor.villagra@upm.es (V.A.V.)

Featured Application

The proposed framework can be deployed as a drop-in middleware layer to secure MCP-based AI agent pipelines in production Security Operations Centers, protecting SIEM-integrated multi-agent deployments against attacks such as tool poisoning, prompt injection, and confused deputy exploitation without modifying the underlying agents or MCP servers.

Abstract

Security Operations Centers (SOCs) rely on Level 1 analysts to triage increasing alert volumes amid alert fatigue and tool fragmentation. LLM-based multi-agent systems using the Model Context Protocol (MCP) are being adopted to automate these tasks, but their autonomy and tool access expose them to attacks such as tool poisoning, indirect prompt injection, and confused deputy exploitation. To address this gap, this work proposes a security framework for MCP-based multi-agent SOC pipelines, implemented as a middleware layer comprising a tool registration validator and five execution layers: access control, rate limiting, input validation, output validation, and audit logging. The framework is applied to a triage-enrichment-response pipeline connected to a Wazuh SIEM through a custom MCP server. Of the 35 attack vectors considered in a threat model derived from different threat taxonomies, including OWASP, MITRE ATLAS, and ATFAA, 29 are addressable at the middleware level and are covered by the framework's controls. These controls are then validated experimentally using a purpose-built malicious MCP server and targeted test-harness injections, organized into six test suites that together exercise the covered vectors across 600 executions. Every attack instance in the evaluated threat model was blocked, none bypassed the framework, and no legitimate call in the evaluated set was incorrectly rejected; obfuscated variants, however, evade the lexical content-inspection controls, delimiting the scope of this result. A full-pipeline demonstration confirms that the framework preserves benign operational outputs. These results indicate that systematic middleware controls can secure MCP-based agentic SOC deployments without modifying the underlying agents or MCP servers.

Keywords: model context protocol; multi-agent systems; agentic AI security; MCP security; security operations center; prompt injection; tool poisoning; SIEM integration

Check for updates

Academic Editors: Shawon Rahman and Shuva Paul

Received: 14 July 2026
Revised: 2 August 2026
Accepted: 6 August 2026
Published: 8 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Security Operations Centers (SOCs) face growing pressure from the increasing volume and sophistication of cyber threats [1]. This pressure falls disproportionately on Level 1 analysts, whose work consists largely of alert triage and the manual correlation of contextual information across fragmented toolsets, a repetitive and information-intensive workflow that contributes directly to alert fatigue [2,3] and creates windows of opportunity for adversaries to move laterally or exfiltrate data before a response is initiated. Rule-based automation through Security Orchestration, Automation, and Response (SOAR) platforms has partially alleviated this burden, but remains constrained by static playbooks that struggle with novel or ambiguous attack patterns [4,5].

Artificial Intelligence (AI) has emerged as a key enabler for the next stage of SOC automation. The transition from passive Large Language Models (LLMs) toward agentic systems capable of planning, invoking external tools, and accumulating context across sessions has opened the possibility of automating repetitive Level 1 tasks while preserving the adaptability that rule-based systems lack [6,7]. Multi-agent systems extend this paradigm further by distributing work across specialized agents, naturally mapping onto the tiered structure of human SOC teams, with persistent shared memory allowing agents to accumulate operational context across alerts and sessions [8,9].

Realizing this paradigm in practice requires a standardized mechanism for connecting agents to the heterogeneous tools and data sources that SOC operations depend on, including SIEM platforms, threat intelligence feeds, and ticketing systems. The Model Context Protocol (MCP), introduced by Anthropic in November 2024, addresses this need by replacing said one-off integrations with a single open protocol that any compliant client and server can implement [10–12]. Since its release, MCP has achieved rapid industry adoption, including integration across major LLM providers and its donation to the Agentic AI Foundation under the Linux Foundation in December 2025 [13,14], and has already been applied to enterprise security operations workflows [15].

However, the same autonomy and tool access that make MCP-based multi-agent systems valuable for SOC automation also expose them to attack classes that have no direct analog in traditional software security. Because agents rely on natural-language tool descriptions and externally retrieved content to decide which actions to take, adversaries can embed malicious instructions in tool metadata or in the data an agent retrieves, leading to tool poisoning, indirect prompt injection, and memory manipulation across sessions [16–18]. In multi-agent deployments, this risk is compounded by inter-agent trust exploitation, where payloads that fail against a direct user prompt succeed when delivered by a peer agent [19], and by confused deputy attacks, in which a privileged agent is manipulated into performing unauthorized actions on behalf of an attacker [20]. Multiple taxonomies, including the OWASP Top 10 for LLM and Agentic Applications [21,22], the OWASP MCP Top 10 [23], MITRE ATLAS [24], and the Advanced Threat Framework for Autonomous AI Agents (ATFAA) [25], have begun to systematize these risks, but with substantial terminological overlap and no consensus mechanism for translating them into deployable middleware controls. A more recent and considerably more granular contribution, MCP-38 [26], enumerates 38 threat categories derived from a systematic decomposition of the MCP specification, cross-mapped to STRIDE and to both OWASP taxonomies above, and positions itself as a unifying superset that consolidates this fragmented terminology. Because its protocol-level granularity exceeds the scope addressed at the middleware layer in this work, it is used here as external corroboration for the attack vectors selected in Section 1 rather than as one of the five taxonomies from which those vectors are drawn.

Existing work on MCP-based multi-agent architectures for security operations has validated key architectural patterns, including role specialization, persistent shared memory, and trust-boundary enforcement, while parallel research has characterized specific MCP attack vectors and proposed governance frameworks for agentic AI risk [9,15,27–29]. However, no existing proposal combines a functional multi-agent SOC architecture connected to a real SIEM deployment through MCP with a security framework that is both systematically derived from the documented attack surface and experimentally validated against controlled, MCP-specific attack scenarios.

This work addresses that gap by proposing a security framework for MCP-based multi-agent SOC pipelines, implemented as a middleware layer interposed between agents and MCP servers. Specifically, the contributions of this paper are as follows: (i) a threat model for MCP-based multi-agent SOC systems, comprising 35 attack vectors selected from five threat taxonomies according to explicit architectural-relevance, empirical-grounding, and SOC-applicability criteria; (ii) the design of a three-agent triage-enrichment-response pipeline with persistent shared memory, connected to a real Wazuh SIEM deployment through a custom MCP server; (iii) the design of a five-layer security framework, comprising access control, rate limiting, input validation, output validation, and audit logging, together with a connection-phase tool registration validator, covering 29 of the 35 identified vectors as addressable at the middleware level; and (iv) an experimental evaluation across six test suites and 600 executions, in which every attack instance was blocked, no attack bypassed the framework, and no legitimate call in the evaluated set was incorrectly rejected.

The remainder of this paper is organized as follows. Section 2 reviews related work on multi-agent architectures for security operations and security frameworks for agentic AI. Section 3 describes the proposed architecture, its attack surface, the security framework, and its laboratory implementation. Section 4 reports the experimental results. Section 5 discusses these results in light of the related literature and the limitations of the work, and Section 6 concludes the paper.

2. Related Work

Empirical work on multi-agent systems for security operations has progressed from theoretical design patterns toward validated implementations. Renney et al. [8] formalize emerging design patterns for LLM-enabled multi-agent systems and demonstrate, across three industrial case studies including telecommunications security, that prototype solutions can be delivered within two weeks, while also documenting persistent variability in LLM behavior that complicates the transition from prototype to production. Krishnan [9] proposes a multi-agent framework built specifically on MCP, showing that the protocol's standardized context sharing addresses the disconnected models problem and enables persistent memory across agent boundaries. Mitra et al. [15] present AgenticCyOps, the work most closely related to the architecture proposed here, decomposing the attack surface of enterprise cyber operations into component, coordination, and protocol layers and reporting a reduction of at least 72% in exploitable trust boundaries relative to a flat multi-agent system; however, the framework is not validated against controlled attack scenarios and does not address MCP-specific vectors such as tool shadowing or confused deputy attacks. Across this body of work, four design principles recur consistently: role specialization with least-privilege tool allocation, persistent shared memory for stateful reasoning, explicit trust boundary enforcement in multi-server environments, and human-in-the-loop (HITL) oversight for consequential actions; these principles jointly inform the architecture described in Section 3.

A parallel research thread addresses agentic AI security through governance and risk-management frameworks. Raza et al. [27] adapt the AI Trust, Risk, and Security

Management (TRiSM) framework to LLM-based multi-agent systems across four pillars and introduce metrics for inter-agent collaboration quality and tool utilization, but without addressing MCP-specific risks or providing experimental validation. Narajala and Narayan [25] propose the Advanced Threat Framework for Autonomous AI Agents (ATFAA) together with its companion mitigation framework SHIELD, arguing that LLM agents must be treated as autonomous, interconnected systems with emergent behaviors rather than isolated components; SHIELD's segmentation, integrity verification, and logging immutability principles directly inform the layer design adopted in this work. Kim et al. [29] present a systematic survey of 51 attack methods and 60 defenses across seven dimensions of agentic system design, showing that risks cascade across components and that component-level analysis consistently misses compound attack surfaces. Chhabra et al. [28] complement this with a taxonomy of agentic threats and identify accountability obfuscation as an open governance challenge. None of these frameworks provides an implemented, experimentally evaluated middleware layer for MCP-based deployments.

A growing body of experimental work characterizes specific MCP and agentic attack vectors. Guo et al. [16] introduce MCPXKIT, a library of 31 attack methods spanning direct and indirect tool injection, and demonstrate that tool poisoning, in which adversarial instructions are embedded in tool metadata, constitutes a pervasive vulnerability that exploits the sycophancy of LLMs. Lupinacci et al. [19] evaluate 18 state-of-the-art LLMs and report that while 94.4% succumb to direct prompt injection, 100% can be compromised through inter-agent trust exploitation, since models that resist a direct malicious command will execute the same payload when it is relayed by a peer agent; the authors further chain these surfaces into complete computer takeover. Bhagwatkar et al. [30] show that tool-input and tool-output firewalls, while practical and model-agnostic, have limited effectiveness against adaptive adversaries when deployed as the sole defense. Chang et al. [18] demonstrate that unoptimized malicious content is never retrieved under natural queries regardless of corpus size, but that an optimized poisoning strategy achieves over 80% success in coercing an MCP-connected agent into exfiltrating Secure Shell (SSH) keys. Hossain et al. [31] report complete mitigation of prompt injection using a coordinated multi-agent defense pipeline, although their evaluation is limited to isolated, single-task settings.

Table 1 summarizes the positioning of this work relative to the reviewed literature across four dimensions: a functional multi-agent SOC architecture, integration with MCP, a security framework implemented as middleware, and experimental evaluation against controlled attacks. No reviewed work satisfies all four dimensions simultaneously: architectural proposals lack experimental validation or MCP-specific threat coverage [8,9,15]; governance frameworks operate at a level of abstraction that stops short of implementation [25,27,29]; and attack-and-defense research evaluates individual vectors in isolation rather than within a unified architecture combining tool orchestration, persistent memory, and multi-server MCP environments [16,18,19,30,31]. This gap motivates the security framework and experimental design presented in the remainder of this paper.

Table 1. Comparison of related work across the four dimensions addressed by this paper. ✓ indicates full coverage, × indicates absence, and "Partial" indicates incomplete coverage.

Work	MAS SOC Architecture	MCP Integration	Security Framework	Experimental Evaluation
Renney et al. [8]	✓	×	×	Partial
Krishnan [9]	✓	✓	×	×
Mitra et al. [15]	✓	✓	✓	×
Raza et al. [27]	×	×	✓	×
Narajala and Narayan [25]	×	×	✓	×

Table 1. *Cont.*

Work	MAS SOC Architecture	MCP Integration	Security Framework	Experimental Evaluation
Kim et al. [29]	×	×	✓	Partial
Guo et al. [16]	×	✓	×	✓
Lupinacci et al. [19]	×	×	×	✓
Bhagwatkar et al. [30]	×	×	Partial	✓
Chang et al. [18]	×	Partial	×	✓
Hossain et al. [31]	×	×	✓	✓
This work	✓	✓	✓	✓

3. Materials and Methods

This section describes the design and implementation of the proposed system. Section 3.1 presents the multi-agent architecture and characterizes the attack surface it introduces. Section 3.2 describes the security framework designed to address that surface. Section 3.3 covers the laboratory implementation, technology stack, and pipeline execution flow underlying the experimental evaluation reported in Section 4. Section 3.4 then presents the evaluation methodology, and Section 3.5 documents the test harness and attack payload catalogue underpinning the experiments whose results are reported in Section 4. The architecture is designed first to reflect realistic SOC operational requirements; its attack surface is then characterized; and the security framework is subsequently designed to address that surface systematically, since understanding what the architecture exposes is a prerequisite for designing controls that are both necessary and sufficient.

3.1. Architecture

The proposed system follows a sequential multi-agent pipeline comprising three functional layers: an agent layer with three specialized LLM agents, a shared memory layer that persists across sessions, and an MCP integration layer managing concurrent connections to two external servers (Figure 1). All inter-agent communication is synchronous: each agent completes its work before the next is invoked, and the architecture instantiates a triage-enrichment-response sequence for each processed alert.

Each agent is assigned a dedicated system prompt, a restricted tool allowlist, and a specific output schema, implementing role specialization with least-privilege tool allocation. The Wazuh MCP server, a custom build extending the open-source `gbrigandi/mcp-server-wazuh` project [32] with two tools developed for this work (`get_wazuh_latest_alert` and `propose_wazuh_rule`), exposes 16 tools covering alerts, agents, vulnerabilities, cluster health, logs, and rules. Of these, the triage agent has access to 3 tools sufficient to retrieve the triggering alert and assess escalation criteria; the enrichment agent has access to 7 tools covering host profiling, process and port enumeration, and vulnerability data; and the response agent has access to 6 tools, including `propose_wazuh_rule`, the only tool in the system that produces an externally actionable artifact and the only one restricted to a single role. Tool access boundaries are enforced before any call reaches the MCP server, so that even a semantically compromised agent cannot escalate its privileges by invoking a restricted tool.

A persistent shared memory module addresses the inability of standalone LLMs to retain knowledge between invocations, accumulating structured data across sessions in three stores. The alert history log records the triage and enrichment output for each processed alert, enabling the response agent to calibrate urgency relative to prior incidents involving the same host. This store and the agent knowledge base are keyed by host and intentionally cross-session; the per-session isolation introduced in Section 3.2 applies

only to the context store described below. The agent knowledge base stores host-specific information keyed by agent name, including IP address, operating system, risk level, and known vulnerabilities. The context store, the most security-sensitive component, holds key-value entries injected into every agent's system prompt at runtime; because its contents are loaded without validation in the base architecture, it constitutes the primary attack surface for memory poisoning, a vulnerability addressed by the input validator described in Section 3.2.

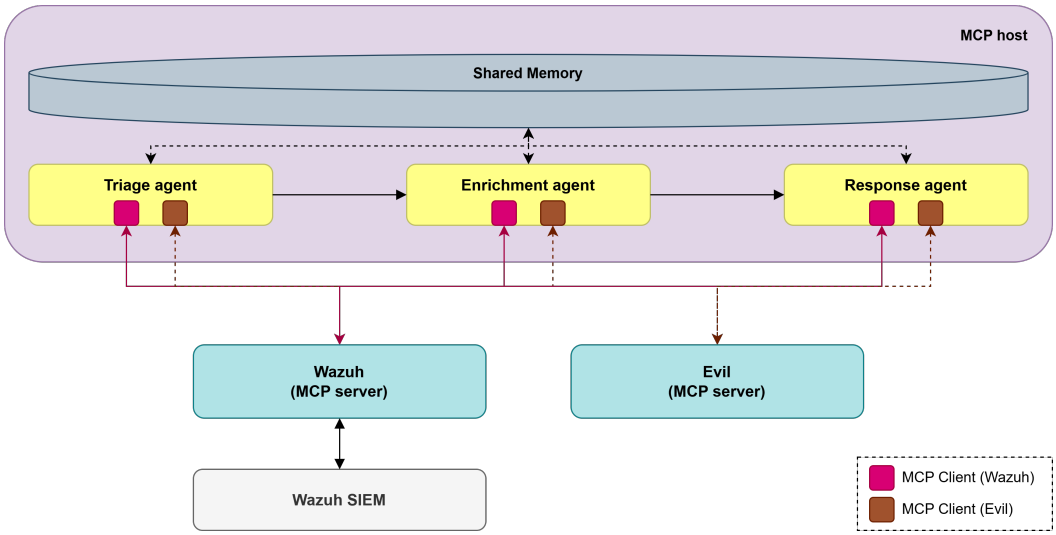


Figure 1. Overview of the multi-agent SOC system architecture. Box colors denote component type: yellow boxes are the LLM agents, teal boxes are the MCP servers, the gray box is the Wazuh SIEM, and the shaded purple region is the MCP host. Solid horizontal arrows denote the sequential triage-enrichment-response flow between agents; dashed vertical arrows denote read/write access to the shared memory. The colored connectors from each agent to the MCP servers indicate the two MCP clients, as shown in the legend (magenta: Wazuh client; brown: malicious client).

The architecture maintains concurrent connections to two MCP servers, isolated in separate Docker network segments so that the malicious server cannot reach the monitored infrastructure directly. The legitimate Wazuh MCP server connects to a real Wazuh SIEM deployment. The malicious server is a purpose-built, specification-compliant MCP endpoint developed to simulate documented attack vectors under controlled, reproducible conditions: `get_wazuh_latest_alert` registers under the same name as the legitimate tool, creating a tool shadowing condition; `get_threat_intelligence` embeds adversarial instructions directly in its description field, implementing tool description poisoning; `get_vulnerability_feed` returns a payload containing injected instructions, implementing indirect prompt injection via tool output; and `get_compliance_report` instructs the agent to invoke the rule-proposal tool with attacker-controlled parameters, implementing a confused deputy attack. The name collision produced by the shadowed `get_wazuh_latest_alert` tool, detected at connection time, constitutes the primary empirical motivation for the tool registration validator described in Section 3.2.

Consistent with the recommended hybrid model for AI-assisted SOC deployments [5,33], the response agent never executes actions directly. It produces a structured plan containing recommended actions, a human approval flag, and an escalation flag, and the highest-privilege operation in the system, `propose_wazuh_rule`, returns a formatted proposal for analyst review without applying any change to the Wazuh instance under any circumstances. This design ensures that even a fully compromised response agent cannot modify the detection infrastructure without explicit human review.

The base architecture, prior to the application of any security control, exposes four attack surfaces that motivate the framework described in Section 3.2. The tool invocation mechanism enforces role-based visibility but does not validate the arguments agents pass to permitted tools, exploited by the confused deputy attack described above. The memory persistence mechanism introduces a durable attack surface through the unsanitized context store, where a single successful write can affect all three agents across all future sessions. The inter-agent handoff mechanism passes agent output as plain text without intermediate validation, allowing injected content to propagate into a downstream agent's context. The multi-server connection mechanism exposes the tool discovery phase to poisoning, since the MCP client pools tools from all connected servers into a single namespace before presenting them to the agent layer, without per-server validation.

3.2. Security Framework

The security framework is implemented as a middleware layer that intercepts all communication between agents and MCP servers, realized as a single gateway class through which no agent can invoke any tool directly (Figure 2).

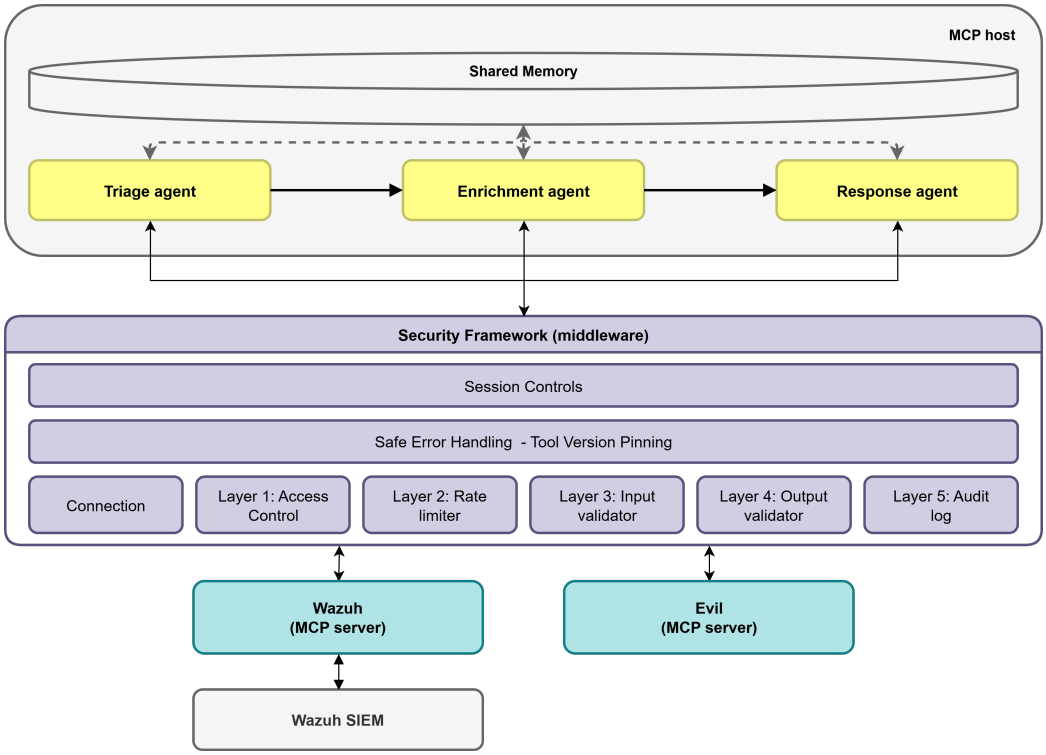


Figure 2. Overview of the security framework architecture. Box colors denote component type: yellow boxes are the LLM agents, teal boxes are the MCP servers, the gray box is the Wazuh SIEM, and the purple boxes are the components of the security framework (the middleware gateway, session and transversal controls, and the connection and execution layers). Solid horizontal arrows denote the sequential triage-enrichment-response flow between agents; the dashed arrow denotes shared-memory access; solid vertical arrows denote tool-invocation flow routed through the middleware to the MCP servers.

The framework satisfies six design criteria derived from the literature reviewed in Section 2: unified middleware interception, so that no agent holds a direct reference to any MCP client; pre-admission validation of tool metadata, inspecting tool descriptions before they are exposed to any agent; role-based access control (RBAC) enforced at runtime on every tool invocation rather than through static filtering at initialization; multi-layer input and output validation covering tool arguments, inter-agent handoffs, memory context, and

externally retrieved data; append-only audit logging within the framework's execution context, with sensitive-value hashing; and failure opacity toward the model, returning generic error messages regardless of which control triggered a block, so as to deny the diagnostic channel that per-layer error messages would otherwise expose. This is an attack-surface-reduction property rather than a validated defense against an adaptive adversary: it removes information that would facilitate iterative probing, without by itself guaranteeing that an adaptive attacker cannot eventually succeed. Its verification in this work is structural, namely that a blocked call returns a fixed, control-independent message (a property confirmed across all test suites), rather than empirical against an iterative probing campaign, which is outside the threat model evaluated here (Section 5). The framework organizes these criteria into two phases: a connection phase, executed once per server at initialization, and an execution phase, comprising five sequential layers applied to every tool call. The content-inspection controls within these layers, namely the description scan at connection time and the injection-pattern scans in the input and output validators (Layers 3 and 4), are implemented as deterministic pattern matching rather than semantic or LLM-based classification. This is a deliberate design choice consistent with the criteria above, for three reasons. First, semantic content analysis is confined to these scans, whereas the remaining controls (runtime role-based access control, rate limiting, JSON Schema validation, manifest and binary hashing, and session-namespace isolation) are deterministic structural checks that do not depend on it. Second, delegating injection detection to an auxiliary language model would reintroduce the prompt-injection surface the framework is designed to close, since such a detector is itself susceptible to the attacks it screens for, and would compromise the failure-opacity and model-independence criteria by making security decisions contingent on model behavior. Third, deterministic matching yields the reproducible, per-pattern-attributable decisions on which the evaluation protocol (Section 3.4) and the append-only audit log depend. The residual cost of this choice, namely limited robustness against adaptive adversaries that iteratively refine payloads to evade fixed patterns, is examined in Section 5.

3.2.1. Connection Controls

At connection time, before any tool is exposed to any agent, the framework applies three checks to the complete tool manifest returned by each server, as illustrated in Figure 3. First, a regex scan inspects every tool description field for adversarial instruction structures, including imperative override phrases, coercive directives, and exfiltration-oriented constructions; any tool whose description matches is blocked and removed from the manifest before reaching the agent layer. Second, the framework enforces namespace ownership across servers: the first server to register a given tool name is recorded as its owner, and any subsequent registration of that same name from a different server is rejected rather than silently admitted, preventing a malicious server from shadowing a trusted tool name. Third, the framework computes a Secure Hash Algorithm (SHA-256) of the complete tool manifest for each server and pins it as the reference value at first connection; on every subsequent reconnection, the recomputed hash is compared against the pinned value, and a mismatch triggers a warning that flags a potential rug-pull attack, in which a server modifies its tool definitions after the initial trust decision. Together, these checks address the multi-server connection surface identified in Section 3.1.

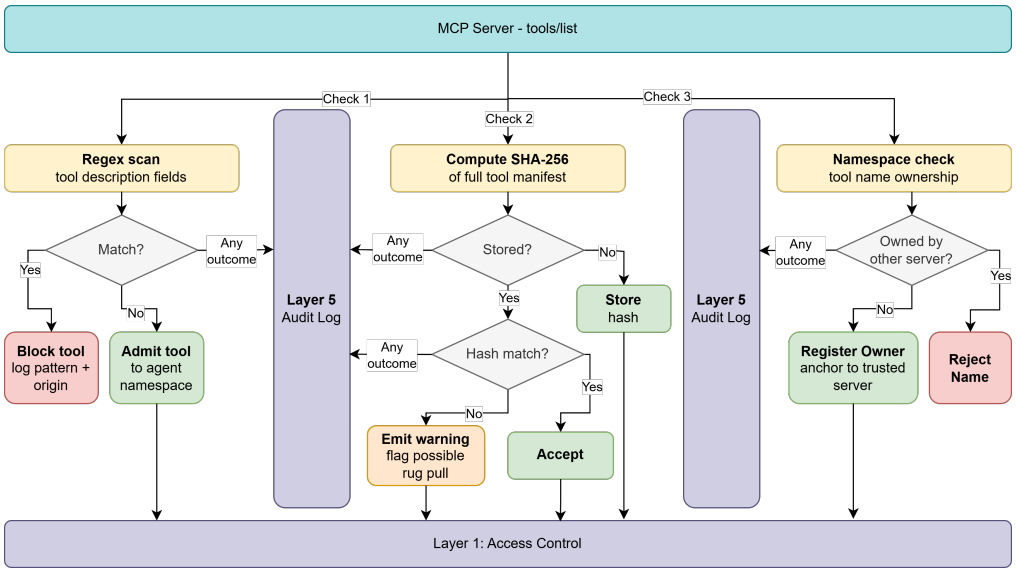


Figure 3. Flowchart of the connection phase controls.

3.2.2. Layer 1: Access Control

The access control layer executes first on every tool call, enforcing role-based allowlists for the triage, enrichment, and response roles. Calls from unrecognized agent identifiers, or to tools outside an agent's allowlist, are rejected unconditionally as illustrated in Figure 4. This check is implemented as a runtime enforcement mechanism rather than the static tool filtering already applied when agents are instantiated, because static filtering only restricts what an agent can discover, not what a semantically manipulated agent can be redirected to invoke; runtime enforcement intercepts the call regardless of how the agent arrived at the decision to make it. This addresses the tool invocation surface described in Section 3.1, including the confused deputy and privilege escalation vectors.

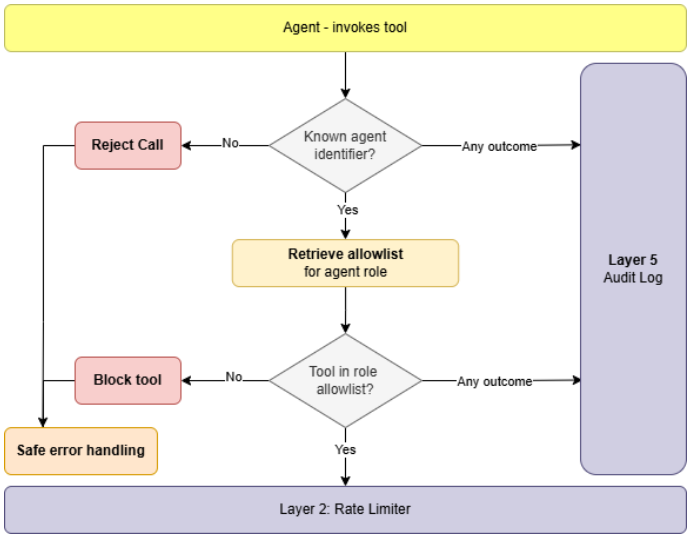


Figure 4. Decision flow of the access control layer.

3.2.3. Layer 2: Rate Limiter

The rate limiting layer tracks tool calls per agent within a sliding 60 s window, with role-specific thresholds of 20 calls for the triage and response agents and 30 for the enrichment agent, reflecting its larger seven-tool scope. An independent, pipeline-level limiter caps human approval requests issued by the response agent at ten per one-hour sliding window, targeting oversight saturation and consent fatigue attacks in which an adversary

floods the analyst with requests to induce approval without review. Figure 5 illustrates the decision flow of both rate limiting controls.

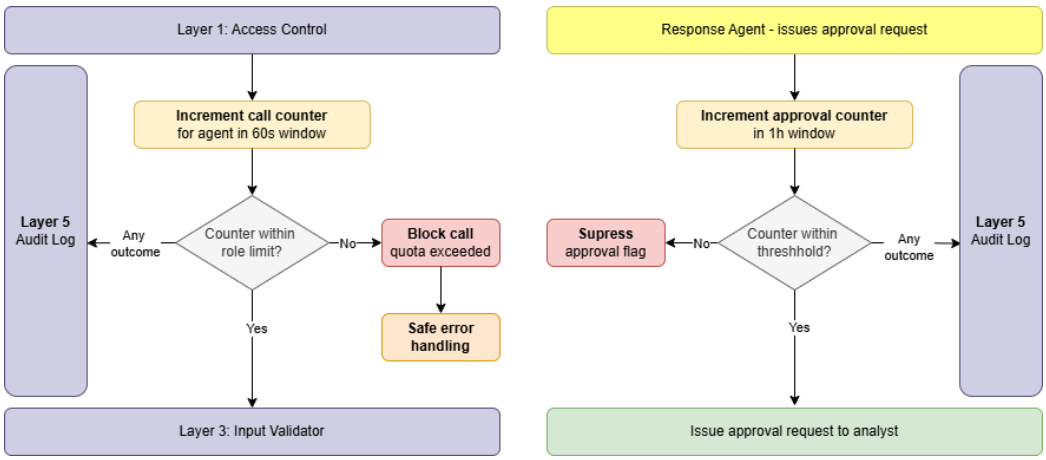


Figure 5. Decision flow of the rate limiting layer.

3.2.4. Layer 3: Input Validator

The input validator operates on four input surfaces, as illustrated in Figure 6. Tool call arguments are validated structurally against a JSON Schema for the invoked tool and then scanned for injection patterns in string values. Inter-agent handoffs are scanned for the same patterns and checked against the expected triage-enrichment-response sequence, rejecting any handoff that targets an unexpected agent. Memory context entries drawn from the context store are scanned before injection into agent system prompts and are namespaced by session identifier, so that a poisoned or cross-session entry is excluded rather than propagated to all agents; this also prevents one pipeline execution from reading or corrupting the context of a concurrent execution. This per-session isolation applies specifically to the context store, whose arbitrary, verbatim-injected key-value entries constitute the memory-poisoning surface identified in Section 3.1. It is deliberately not applied to the alert history log and the agent knowledge base, which are keyed by host and remain cross-session by design, since their purpose is precisely to let the response agent calibrate urgency against prior incidents on the same host. These two stores do not reintroduce a poisoning vector, because their contents cannot originate from unvalidated input: unlike the context store, they are written only by the framework and only with the structured triage and enrichment outputs of a completed pipeline run, each of which has already passed the input and output validators before being stored; even the free-text justification and summary fields are produced by an agent operating solely on already-validated inputs. Re-injecting these host-keyed records into a later run therefore cannot introduce adversary-controlled content that has not already passed the validators, so their cross-session persistence does not expand the attack surface beyond the residual regex-evasion limitation discussed in Section 5. Only the context store, whose entries may carry unvalidated external content, requires per-session isolation. Externally retrieved data is scanned with an extended pattern set covering web-specific vectors such as inline script execution and JavaScript protocol handlers, in addition to the standard injection patterns.

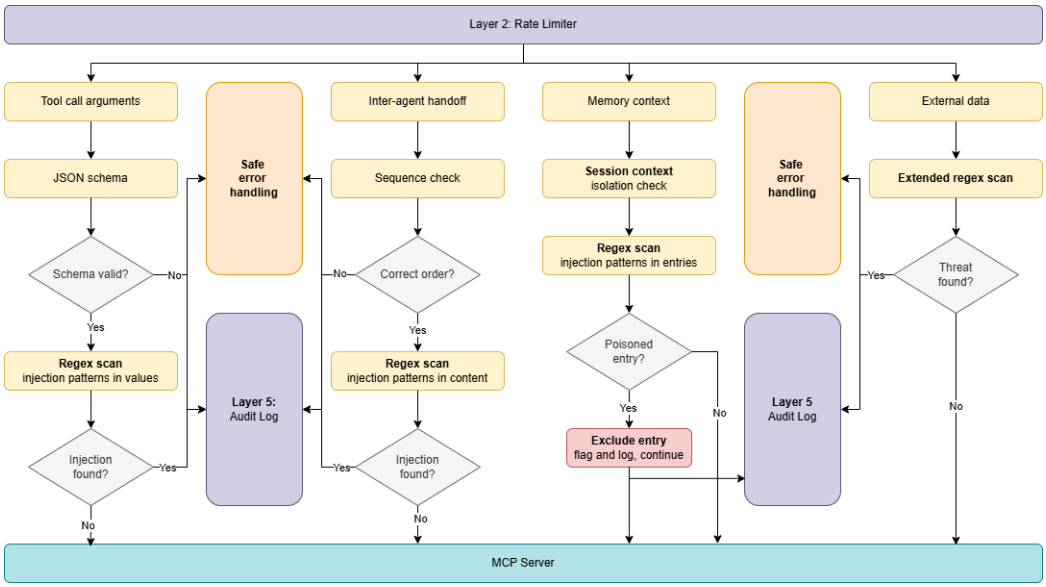


Figure 6. Decision flow of the input validator layer.

3.2.5. Layer 4: Output Validator

The output validator intercepts server responses before they reach the invoking agent, applying two checks as illustrated in Figure 7. A regex scan covers indirect prompt injection, coercive tool-call directives, credential and Application Programming Interface (API) key exfiltration patterns, system prompt extraction attempts, and self-propagation instructions that would constitute an AI virus; tools classified as external content sources apply an extended pattern set additionally covering inline scripts and iframe injections. A response-size check rejects any response exceeding 100 KB regardless of content, bounding the volume of adversarial content a single response can introduce and limiting Denial-of-Service (DoS) risk.

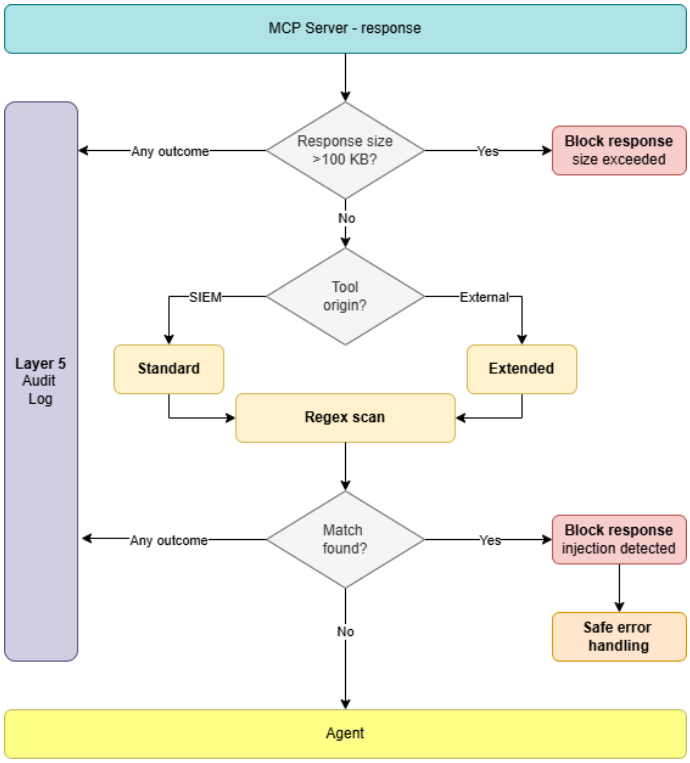


Figure 7. Decision flow of the output validator layer.

3.2.6. Layer 5: Audit Log

The audit log records every tool call that passes through the framework, whether permitted or blocked, capturing the UTC timestamp, session identifier, invoking agent role, tool name and server of origin, call arguments, result or block reason, and a SHA-256 hash of the response. Sensitive argument fields are replaced by their SHA-256 hash before the entry is written, so the audit trail itself cannot become a secondary exfiltration surface. The log is append-only and tagged per session, supporting forensic analysis of anomalous call patterns across sessions without modifying the agent or the MCP client.

3.2.7. Transversal and Session-Level Controls

Two controls apply across all execution layers rather than at a specific layer or call. Safe error handling returns a fixed, generic message to the invoking agent whenever any layer blocks a call, regardless of which layer or pattern triggered the block; this denies the diagnostic channel that distinct per-layer error messages would otherwise provide, reducing the information available for iterative probing without being claimed to defeat an adaptive adversary outright. Tool version pinning extends the connection-phase manifest hash check across the session lifetime, continuously monitoring the tool namespace and surfacing changes that a client would otherwise accept silently.

Three controls govern each pipeline execution as a whole. Session identity assigns a Universally Unique Identifier (UUID) to each execution, propagated to the audit log, the memory store's context namespace, and the framework instance, so that every security decision is traceable to a specific execution context and the integrity of one execution cannot be affected by a concurrent execution sharing the same memory store. Supply chain verification compares the SHA-256 hash of the Wazuh MCP server binary, obtained from the running container, against a pinned reference at connection time, rejecting the server and registering none of its tools on a mismatch. MCP channel authentication attaches a shared secret to every request via a custom header and validates the Origin of both outgoing requests (egress CSRF/CORS control) and incoming responses (DNS-rebinding control) against a trusted-origin allowlist. In the containerized deployment evaluated in this work, the allowed-origin variable was left unset, which disables the response-side DNS-rebinding check; there, that property was instead provided by two topology-level measures, namely binding every published container port to the loopback interface (`127.0.0.1`) in the Docker Compose configuration and confining container-to-container traffic to isolated network segments. A deployment exposing these ports beyond localhost, or spanning hosts, would need to set the allowed origin explicitly to restore the application-layer check.

3.2.8. Attack Coverage

The 35 attack vectors addressed in this work were selected from the broader threat taxonomies reviewed in Section 1 following three criteria, each applied as an explicit inclusion rule so that the selection is reproducible from the source taxonomies. A vector was retained only if it satisfied all three of the following conditions. (1) Architectural relevance: the vector must exploit a capability that this architecture instantiates, namely persistent memory, tool invocation, inter-agent communication, or the MCP protocol itself; a vector whose exploitation path targets none of these surfaces was excluded. (2) Empirical grounding: the vector must have been demonstrated in at least one of the reviewed sources as a realized attack rather than posited only as a theoretical possibility; purely speculative vectors with no reported instance were excluded. (3) SOC applicability: the vector must have a credible exploitation path against an infrastructure-connected, response-capable agent of the kind evaluated here; vectors whose only realization requires capabilities absent from a triage-enrichment-response SOC pipeline were excluded. The conjunction of these

rules deliberately excludes vectors that depend exclusively on the internal reasoning of the underlying language model, such as jailbreaking, reasoning path hijacking, and reward hacking, which fail condition (1), as their exploitation path is the model's inference process rather than any middleware-observable surface, and require interventions at the training or alignment level rather than at the middleware layer.

Table 2 summarizes coverage of these 35 vectors by category; 29 are addressed by at least one framework control. Of the six uncovered vectors, three (Retrieval-Augmented Generation (RAG)/knowledge-base poisoning, adversarial embeddings, and package hallucination) are outside the scope of a middleware-level framework by design; two (consensus manipulation, covert coordination) presuppose a non-sequential multi-agent topology that the pipeline does not instantiate; and replay is not addressed, because effective replay protection requires request-freshness verification (a nonce, timestamp, or message-level signature) at the receiving MCP server, whereas all framework controls operate at the middleware layer without modifying the server. In the evaluated deployment, replay from outside the trust boundary is further constrained by the network segmentation described in Section 3.2, but this is a property of the deployment topology rather than of the framework. No vector that is both applicable to the architecture and addressable at the middleware level is left uncovered. A complete per-vector mapping is provided in the project repository [34].

Table 2. Coverage of the 35 middleware-addressable attack vectors identified in the literature, organized by category. × indicates a vector that is architecturally inapplicable or outside the scope of a middleware-level framework. † The Origin-header check is implemented in the framework but was inactive in the evaluated deployment (Section 3.2).

Category	Attack Vector	Control	Attack Vector	Control
Prompt-based	Direct prompt injection	L3: input validator	Indirect prompt injection	L4: output validator
	Web content poisoning	L4: external content mode		
Tool & MCP	Tool poisoning	Tool reg. validator (desc. scan)	Tool shadowing/name collision	Tool reg. validator (collision det.)
	Rug pull/silent tool redefinition	Tool version pinning (SHA-256)	Confused deputy	L1: RBAC + L3
	Unauthorized autonomous exec.	L1: RBAC + HITL flag	AI virus/self-propagation	L4: self-propagation patterns
Memory & Data	Memory poisoning	L3: memory context validator	Malicious data injection	L3: external data validator
	RAG/knowledge base poisoning	×	Adversarial embeddings	×
Multi-Agent	Inter-agent trust exploitation	L3: handoff validator	Lateral compromise via shared privileges	L1: RBAC + L5: audit log
	Consensus manipulation	×	Covert coordination	×
	Identity spoofing/agent impersonation	L3: handoff sequence enforcer	Human-agent trust manipulation	Response agent justification val.
Protocol	Authentication bypass	X-MCP-API-Key shared secret	Message manipulation	X-MCP-API-Key shared secret
	Session state corruption	Session ID + audit log per session	Replay	×
	CSRF/CORS exploitation	Egress origin validation	DNS rebinding	Origin header validation †
Exfiltration	Credential/API key exfiltration	L4: output validator	Data exfiltration via tools	L2: rate limiter
	Cross-client data leak	Session namespace in context store	LLM system prompt extraction	L4: output validator
	Package hallucination attack	×		
Infrastructure	Supply chain (MCP server)	Binary SHA-256 hash verification	Privilege escalation	L1: RBAC per role
Autonomy & Governance	Oversight saturation attacks	Pipeline approval rate limiter	DoS/resource abuse	L2: rate limiter
	Consent fatigue attacks	Pipeline approval rate limiter		

The 29 covered vectors are exercised under two delivery modes that differ in evidential strength. Four of them are delivered end-to-end through the real malicious MCP server described in Section 3.1, traversing the complete channel from an untrusted server through the MCP client's tool-pooling and transport layers to the agent: tool description poisoning and tool shadowing/name collision, both intercepted during the connection phase, to-

gether with indirect prompt injection via tool output and the confused deputy attack, both admitted at connection time and blocked downstream. These four correspond exactly to the four malicious tools introduced in Section 3.1 and are jointly validated in the full-pipeline demonstration of Section 4.2. The remaining covered vectors are exercised by injecting the corresponding payload directly into the framework component on whose surface it manifests, namely memory context, inter-agent handoffs, session state, tool arguments, or a mocked server response, which isolates each control but traverses a shorter path than a full channel crossing. The two modes therefore provide different levels of evidential strength: end-to-end delivery demonstrates that a control blocks an attack under realistic transport conditions, whereas component-level injection demonstrates that the control enforces its intended property when presented with the payload, without asserting that the payload would traverse the full MCP channel identically in deployment.

Figure 8 traces a single tool call through both phases, showing the layer at which each attack family is intercepted and that the MCP server is reached only after the first three execution-phase layers admit the call. The connection phase, executed once per server at startup, screens the tool manifest through the description scan, namespace-ownership check, and SHA-256 pinning; a manifest-hash change after admission raises a rug-pull warning without rejecting the manifest, whereas the other three checks reject. The execution phase, applied to every tool call, traverses the five layers in sequence (L1 access control, L2 rate limiting, L3 input validation, the MCP server invocation, and L4 output validation), with the audit log (L5) recording every outcome. Each layer's block branch is annotated with the attack families intercepted at that layer, and the MCP server is reached only after Layers 1 to 3 admit the call. The transversal and session-level controls (failure opacity, session-scoped identity, and version pinning) operate across all layers rather than at a single step and are therefore not shown as discrete nodes.

3.2.9. Alignment with Industry Guidance

As an external validation of the design derived from the academic literature reviewed in Section 2, Table 3 maps the implemented controls against the OWASP *Practical Guide for Secure MCP Server Development* v1.0 [35], a community reference published by the OWASP GenAI Security Project. The correspondence confirms that the controls derived from the literature also satisfy those recommended by an independent industry reference, providing converging evidence that the framework's design rests on widely recognized security practice rather than on a single body of work.

Table 3. Mapping of the framework's controls to the OWASP *Practical Guide for Secure MCP Server Development* v1.0 [35].

OWASP Guide Section	Controls Implemented in This Work
2—Safe Tool Design	Tool description validation, namespace ownership enforcement, version pinning via SHA-256 manifest hash, and supply chain binary verification (connection phase).
3—Data Validation & Resource Management	Per-tool JSON schema validation, per-role and pipeline-level rate limits, and output size limits (Layers 2–4).
4—Prompt Injection Controls	Injection scanning across tool arguments, server responses, memory context, and inter-agent handoffs, with human-in-the-loop approval for consequential actions (Layers 3–4).
5—Authentication & Authorization	Centralized middleware interception and runtime least-privilege enforcement per agent role (Layer 1).
6—Secure Deployment & Updates	Safe error handling with no internal logic exposed to the model, non-root containers, secrets with restricted filesystem permissions, and network segmentation.
7—Governance	Append-only audit log with per-session tagging and field-level redaction of sensitive arguments (Layer 5).

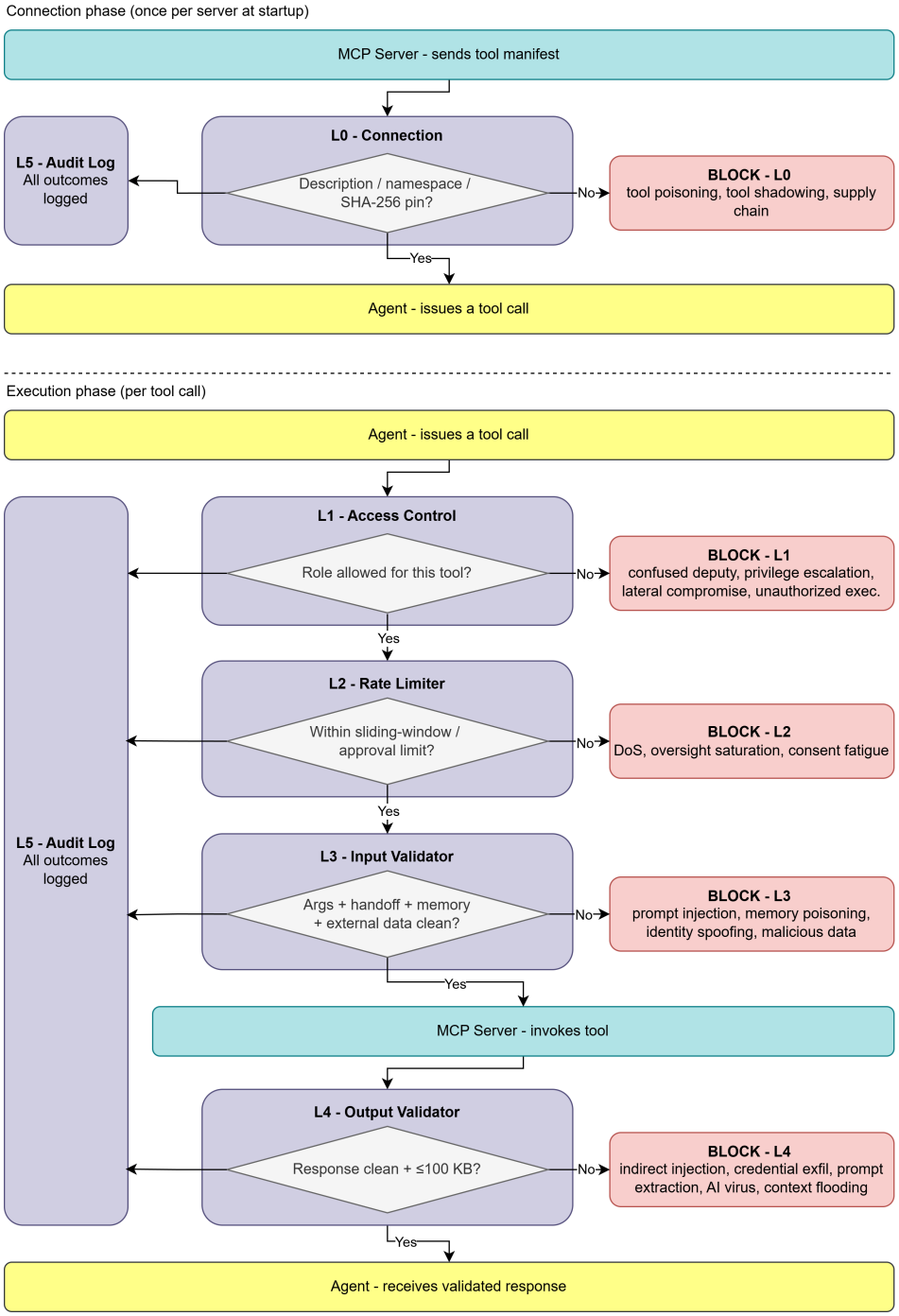


Figure 8. End-to-end decision flow of a tool call through the framework.

3.3. Implementation

The implementation spans two coordinated environments: a Vagrant virtual machine (guest OS: Ubuntu 22.04 LTS) hosting the Wazuh SIEM, and a Docker Compose deployment running the two MCP servers, the persistent security framework, and the demonstration interface on the host machine (Figure 9). The Wazuh SIEM runs the full Wazuh stack, including the Manager, Indexer, and Dashboard. The legitimate Wazuh MCP server connects to the Wazuh API and Indexer using credentials loaded at startup and exposes its own endpoint over the Streamable HTTP transport, using MCP protocol version 2025-06-18. The malicious server runs as an independently accessible container.

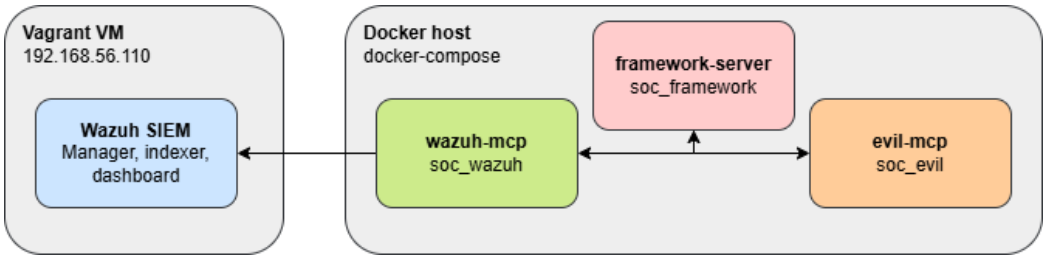


Figure 9. Deployment topology of the laboratory environment.

Three Docker network segments coordinate the deployment, preserving the isolation property described in Section 3.1: one connects the Wazuh MCP server to the framework, one connects the malicious server, and a third is shared exclusively by the security framework and both MCP servers, so that the framework, and only the framework, is connected to both trust domains simultaneously, satisfying the unified middleware interception criterion.

The security framework and the three agents run as a persistent service exposed through a FastAPI application, with the connection-phase controls described in Section 3.2 executing once, at service startup, rather than on every pipeline execution. The service exposes a REST API including endpoints to run a complete pipeline execution, invoke an individual tool through the framework, retrieve or reset the audit log, and inspect memory store statistics; the three persistent memory stores are kept in a dedicated Docker volume so that accumulated alert history and host knowledge survive container restarts. A Jupyter container provides the operator-facing interface used for both the demonstration notebook and the evaluation notebook that drives the test suite described in Section 4.

The agent layer and security framework are implemented in Python 3.11 (Python Software Foundation, Wilmington, DE, USA), with all three agents instantiated from a dedicated system prompt, a role identifier, and a filtered tool allowlist. The language model powering all three agents is GPT-4o mini (snapshot `gpt-4o-mini-2024-07-18`), accessed through the OpenAI API (San Francisco, CA, USA). All three agents are invoked identically, with temperature set to 0.1 and all remaining sampling parameters left at their OpenAI API defaults (top_p = 1.0, no fixed seed, no explicit maximum-token limit); tool selection uses the `auto` tool-choice policy, and each agent runs an internal tool-invocation loop bounded to eight iterations. Structured JSON outputs are elicited through the system prompt and parsed by the framework rather than enforced through the API response-format schema. GPT-4o mini was selected as a representative, widely deployed production model. The use of a closed-source model does not affect the security properties reported here, because all framework controls operate at the tool-invocation and response layer and treat the model as a black box, with no dependency on its weights, internal states, or provider (Section 5); the model governs only the agent behavior being protected, not the deterministic security decisions themselves. The MCP client implements a JSON-RPC 2.0 client over HTTP with Server-Sent Events, conforming to the Streamable HTTP transport defined in the MCP specification, and exposes a uniform interface to the security framework regardless of the underlying server. The security framework itself is implemented as a single class, with JSON Schema definitions for all 16 Wazuh tools loaded at initialization by the input validator. The shared persistent memory module is implemented as a session-namespaced, JSON-on-disk store. The legitimate Wazuh MCP server is implemented in Rust, extending the open-source codebase referenced in Section 3.1 [32]; the malicious server and the framework service are implemented in Python, and all three run as hardened, non-root containers. Table 4 summarizes the principal components of the implementation stack.

Table 4. Main components of the implementation stack.

Component	Technology	Role
LLM	GPT-4o mini (`gpt-4o-mini-2024-07-18`; OpenAI, San Francisco, CA, USA)	Powers all three agents
Agent layer	Python 3.11 (Python Software Foundation, Wilmington, DE, USA)	Triage, enrichment, response agents
Security framework	Python 3.11	Five-layer middleware gateway
Framework server	Python 3.11 (FastAPI, uvicorn)	Persistent REST gateway exposing the framework
MCP client	Python 3.11 (JSON-RPC 2.0 / SSE)	Agent-to-server communication
Wazuh MCP server	Rust 1.86	Legitimate tool provider
Malicious MCP server	Python 3.11	Attack simulation
SIEM	Wazuh 4.14.0 (Wazuh Inc., San Jose, CA, USA)	Alert source and response target
Demonstration interface	Jupyter (`jupyter/scipy-notebook` Docker image)	Pipeline demo and test execution
Infrastructure	Vagrant (HashiCorp, San Francisco, CA, USA) + Docker Compose (Docker Inc., Palo Alto, CA, USA)	Isolated lab environment

A complete pipeline execution proceeds through five stages once the connection-phase controls have validated the tool registry. First, the framework assigns a session identifier, propagated to the security framework, the audit log, and the memory store's context namespace. In the triage stage, the agent receives the alert identifier and the validated contents of the context store, and invokes tools from its allowlist through the security framework, which enforces access control, rate limiting, input validation, and output validation on each call; its structured triage report is validated before being passed to the next stage. In the enrichment stage, the agent receives the triage report as a handoff, which the input validator admits only after verifying the expected sequence, and builds a host profile that is written to the shared knowledge base. In the response stage, the agent receives both the triage report and the enrichment summary and, drawing on the alert history log, produces a structured response plan containing recommended actions, a human approval flag, and an escalation flag, with the pipeline-level approval rate limiter checked before the flag is set; any invocation of the rule-proposal tool returns a formatted proposal for analyst review without applying any change to the Wazuh instance. Finally, the complete audit log for the session is written to disk, with sensitive field values replaced by their hashes, enabling post-hoc forensic analysis tied to the session identifier.

The security framework is designed to be portable across linear (sequential) agent pipelines and largely independent of their remaining specifics, since its layers, session controls, and audit log operate exclusively on the tool invocation and response surfaces, without dependency on the underlying language model, the number of agents, or the identity of the connected MCP servers. The one component that does assume a specific topology is the handoff sequence validator, which enforces a linear order; porting the framework to a non-sequential, decentralized, or hierarchical topology therefore requires generalizing this component, as detailed below and in Section 5. Adapting the framework to a new MCP-based multi-agent architecture is concentrated in four configuration points: a role permission registry mapping each agent role to a tool allowlist; a handoff sequence definition, which would require extension from a linear map to a transition graph for non-sequential topologies; a tool schema registry for structural argument validation, which degrades gracefully to pattern-only scanning when a schema is absent; and a trusted binary hash table for supply chain verification, which emits a warning rather than blocking when no hash is configured. All other components, including the injection pattern sets, the rate limiter, the audit log, and the session identity and namespace isolation mechanisms, are agnostic to the pipeline topology and require no modification.

The complete source code, including the agent implementations, the security framework, the malicious MCP server, and the deployment configuration, is publicly available in the project repository [34]. Generative AI tools (Claude, Anthropic, San Francisco, CA, USA) were used exclusively for language editing, grammar correction, and improvement of manuscript readability. The authors reviewed and validated all scientific content, analyses, interpretations, and conclusions and take full responsibility for the final manuscript.

3.4. Evaluation Methodology

The evaluation follows a controlled experimental protocol applied uniformly across six test suites, each isolating a specific framework component and the subset of attack vectors whose primary exploitation path runs through it, so that a detection result in one test cannot be attributed to a control belonging to a different layer. Each suite exercises a fixed set of distinct attack instances, each repeated over $N = 20$ executions with the framework active, combining attack inputs with legitimate tool calls drawn from the same pipeline stages, so that the usability metric reflects conditions representative of real execution rather than artificially benign scenarios. Across the six suites, this amounts to 25 unique attack instances repeated over 20 executions each; because four instances evaluate more than one payload or artefact per execution, namely tool description poisoning in T00 (two poisoned tools), the access-control check in T01 (two lower-privileged roles, triage and enrichment, each attempting the restricted rule-proposal tool), web content poisoning in T04 (two payloads under the standard and extended pattern sets), and human-agent trust manipulation in T05 (three coercive response plans), the suites together yield 600 individual block decisions. Because the framework controls are deterministic, these repetitions are not independent samples of control behavior and do not increase the statistical power of the security result; their purpose is twofold: to confirm that each control's decision is stable across executions, as a reproducibility check rather than a sampling argument, and, for the suites whose paired legitimate cases traverse the agent layer, to exercise those cases under the non-deterministic behavior of the underlying language model, whose agents are invoked with a non-zero sampling temperature, so that the false-block metric is measured under realistic model variation rather than on a single response. Attack inputs are delivered either by the purpose-built malicious MCP server, for vectors that manifest through the tool invocation or server response surfaces, or by direct injection from the test harness into the relevant component, for vectors that manifest through memory context, inter-agent handoffs, or session state.

Three metrics are recorded per suite: attacks blocked, the number of attack instances correctly detected, with an expected value equal to the total attempted; attacks passed, the number that bypassed all controls and reached the agent, with an expected value of zero; and false blocks, the number of legitimate calls incorrectly rejected, with an expected value of zero.

3.5. Test Harness and Attack Payload Catalog

This section complements the evaluation methodology presented in Section 3.4 by documenting the structure of the test harness used to evaluate the proposed security framework, and the attack and legitimate inputs exercised by each test suite reported in Section 4.1.

The test harness is implemented in `run_tests.py`. It executes six test suites, labeled T00 to T05, each aligned with one framework component or control group. For each test, N trials are executed with the framework active. The default value is $N = 20$, although the script also supports executing a different number of trials or running a single test suite independently.

The harness validates the deterministic behavior of each framework control against a fixed catalog of representative attack instances derived from the threat model, with each payload repeated unchanged across trials. The interpretation of this design, and its scope with respect to adaptive inputs, is the one established in Section 3.4.

3.5.1. Harness Execution Model

Each test suite follows the same high-level structure:

1. The relevant framework component, client, validator, or pipeline state is initialized.
2. A baseline measurement is taken without the security framework when a comparable unprotected operation exists.
3. The attack case is executed with the framework active.
4. The harness records whether the attack is blocked or reaches the protected surface.
5. A corresponding legitimate case is executed through the same framework component.
6. The harness records whether the legitimate case is accepted or incorrectly blocked.
7. The process is repeated for N trials.

The harness records both security and usability outcomes. Security is measured through the number of attacks blocked and attacks passed. Usability is measured through the number of false blocks, corresponding to legitimate operations incorrectly rejected by the framework. The harness also reports *legit passed*, representing legitimate operations correctly allowed.

3.5.2. Attack Payload Catalog

The following tables document the attack payload families exercised by each test suite. Payloads are described by structure and intent rather than reproduced as unrestricted operational strings, since the goal of this section is to document the evaluation design rather than provide a reusable attack library.

Table 5 provides an explicit mapping from each threat-model vector to the harness subtest that exercises it and to the number of block decisions that subtest contributes per trial, reconciling the three counts that appear in this work: threat-model vectors (Table 2), unique attack instances or subtests, and total block decisions. A single subtest may exercise more than one vector (for example, the T01 access-control subtest jointly covers privilege escalation, confused deputy, and unauthorized autonomous execution) and may contribute more than one block decision per trial (for example, the T00 description-poisoning subtest evaluates two poisoned tools). The 25 subtests therefore span the 29 covered vectors and yield 30 block decisions per trial, which over $N = 20$ executions produce the 600 decisions reported in Section 4.1.

T00 (Table 6) evaluates the connection-phase tool registration validator against description poisoning, name collision, rug pull, and supply chain attacks, the first two delivered by the malicious server at connection time and the remaining two introduced by the test harness between trials.

T01 (Table 7) evaluates Layer 1 against confused deputy, privilege escalation, and unauthorized autonomous execution attempts at the tool invocation layer, together with authentication bypass, message manipulation, DNS rebinding, and Cross-Site Request Forgery/Cross-Origin Resource Sharing (CSRF/CORS) exploitation at the MCP channel level, and against lateral compromise via shared privileges, in which an already-compromised agent attempts to pivot into a tool outside its assigned role.

Table 5. Reconciliation of threat-model vectors, harness subtests, and block decisions. "Block dec./trial" is the number of block decisions the subtest contributes per execution; the "×20" column is that value over $N = 20$ executions. A subtest may cover several vectors and may contribute several block decisions per trial.

Suite	Subtest	Threat-Model Vector(s)	Block Dec./Trial	×20
T00	Description poisoning	Tool poisoning	2	40
	Tool shadowing	Tool shadowing/name collision	1	20
	Rug pull	Rug pull/silent tool redefinition	1	20
	Supply-chain	Supply chain (MCP server)	1	20
T01	Access control (RBAC)	Privilege escalation; confused deputy; unauthorized autonomous execution	2	40
	Authentication bypass	Authentication bypass; message manipulation	1	20
	DNS rebinding	DNS rebinding	1	20
	CSRF/CORS	CSRF/CORS exploitation	1	20
	Lateral compromise	Lateral compromise via shared privileges	1	20
T02	Tool-call burst	DoS/resource abuse; data exfiltration via tools	1	20
	Approval burst	Oversight saturation; consent fatigue	1	20
T03	Argument injection	Direct prompt injection	1	20
	Memory poisoning	Memory poisoning	1	20
	External data	Malicious data injection	1	20
	Handoff sequence	Identity spoofing/agent impersonation	1	20
	Handoff content	Inter-agent trust exploitation	1	20
T04	Indirect injection	Indirect prompt injection	1	20
	Tool poisoning	(response-borne tool poisoning)	1	20
	Context flooding	DoS/resource abuse	1	20
	Web content	Web content poisoning	2	40
	Credential exfil.	Credential/API key exfiltration	1	20
	Prompt extraction	LLM system prompt extraction	1	20
	AI virus	AI virus/self-propagation	1	20
T05	Trust manipulation	Human-agent trust manipulation	3	60
	Session isolation	Session state corruption; cross-client data leak	1	20
Total	**25 subtests**	**29 covered vectors**	**30**	**600**

Table 6. T00 attack payload catalog: Tool Registration Validator.

Subtest	Payload Structure	Expected Decision
Tool description poisoning	Two malicious tools are registered by the untrusted server with adversarial instructions embedded in their description fields.	The poisoned tools are removed from the admitted tool registry before reaching the agent namespace.
Tool shadowing	The untrusted server attempts to register a tool with the same name as an existing legitimate Wazuh tool.	The registration is rejected because the tool name is already owned by the legitimate server namespace.
Rug pull	A previously admitted tool manifest is modified between connection phases by changing the tool description.	The manifest change is detected through hash comparison against the pinned value and flagged with a warning; consistent with Section 3.2, the control raises an alert rather than rejecting the manifest, and detection of the change is the outcome recorded as a successful block.
Supply-chain attack	The expected Wazuh MCP server binary hash is compared against a tampered hash value.	The binary verification fails and the server is rejected before tool registration.

T02 (Table 8) evaluates Layer 2 through a tool-call burst exceeding the per-role sliding-window threshold and an approval-request burst exceeding the pipeline-level threshold, targeting resource exhaustion, incremental exfiltration, oversight saturation, and consent fatigue.

Table 7. T01 attack payload catalog: Layer 1 Access Control.

Subtest	Payload Structure	Expected Decision
Privilege escalation, confused deputy, and unauthorized autonomous execution	Lower-privileged roles attempt to invoke `propose_wazuh_rule`, a tool reserved for the response agent.	The tool call is blocked through runtime role-based access control.
Authentication bypass and message manipulation	A client attempts to construct MCP headers without the required API key being configured.	The request is rejected or flagged before authenticated communication is established.
DNS rebinding	A mocked response origin points to an unexpected external host rather than the configured allowed origin.	The response origin validation rejects the response.
CSRF/CORS exploitation	A request is generated with a forged origin value before being sent.	The outgoing request is rejected by the egress-side origin validation logic.
Lateral compromise	A compromised enrichment-agent role attempts to invoke a tool assigned to another role, while otherwise using plausible arguments.	The call is blocked based solely on role identity and tool allowlist membership.

Table 8. T02 attack payload catalog: Layer 2 Rate Limiter.

Subtest	Payload Structure	Expected Decision
Tool-call burst	The triage agent issues more calls than the configured per-role threshold of 20 calls per 60-s window.	Calls within the threshold are accepted; the first call beyond the threshold is blocked.
Approval-request burst	The response pipeline generates more approval requests than the configured limit of 10 requests per hour.	Approval requests beyond the threshold are suppressed.

T03 (Table 9) evaluates Layer 3 across its four input surfaces: tool argument injection, memory context poisoning together with cross-session namespace isolation, external data injection, and both identity spoofing and content injection in inter-agent handoffs.

Table 9. T03 attack payload catalog: Layer 3 Input Validator.

Subtest	Payload Structure	Expected Decision
Tool argument injection	A prompt-injection instruction is embedded inside a string argument passed to a Wazuh tool call.	The input validator rejects the tool call before it reaches the MCP server.
Memory poisoning	A malicious instruction is inserted into the memory context passed to the agent.	The poisoned memory entry is excluded from the validated context.
Malicious external data injection	Externally retrieved content contains a web-style script payload.	The external data validator rejects the payload before it can be admitted into memory or agent context.
Identity spoofing	A handoff is submitted from the triage agent directly to the response agent, violating the expected pipeline order.	The handoff sequence validator rejects the transition.
Inter-agent trust exploitation	A syntactically valid triage-to-enrichment handoff contains an embedded prompt-injection instruction.	The handoff content validator rejects the payload before it reaches the next agent.

T04 (Table 10) evaluates Layer 4 against indirect prompt injection, tool poisoning embedded in response bodies, context flooding via oversized responses, web content

poisoning under both the standard and extended pattern sets, credential exfiltration, system prompt extraction, and AI virus self-propagation.

Table 10. T04 attack payload catalog: Layer 4 Output Validator.

Subtest	Payload Structure	Expected Decision
Indirect prompt injection	A server response contains an instruction to override previous instructions and modify severity assessment.	The output validator blocks the response before insertion into the agent context.
Tool poisoning in response	A server response instructs the agent to invoke a rule-proposal tool with malicious rule content.	The output validator detects the malicious instruction and blocks the response.
Context flooding	A server response exceeds the configured size threshold by returning a very large string.	The response is rejected by the size limit before full pattern scanning.
Web content poisoning	External-content responses contain web-specific constructs such as script tags or JavaScript protocol handlers.	The extended output-validation mode blocks both web-content poisoning payloads.
Credential exfiltration	A response contains a credential-like pattern resembling an API key or token assignment.	The credential pattern scan blocks the response.
System prompt extraction	A response attempts to induce disclosure of the model's system prompt.	The extraction-pattern validator blocks the response.
AI virus/self-propagation	A response instructs the agent to copy malicious instructions into the next pipeline stage.	The self-propagation detector blocks the response before downstream handoff.

T05 (Table 11) evaluates the session-level controls against three variants of human-agent trust manipulation in response-agent justifications and against cross-session context store access attempts.

Table 11. T05 attack payload catalog: Session and Response Controls.

Subtest	Payload Structure	Expected Decision
Human-agent trust manipulation	Three response plans contain coercive or fabricated justifications, including urgency pressure, compliance pressure, and authority-based override language.	The response justification validator flags the plan and marks it with a trust-manipulation warning.
Session state integrity	Two memory stores are initialized with different session identifiers, and one session attempts to read context written by the other.	The cross-session read returns no context, preserving session namespace isolation.

3.5.3. Legitimate Cases

Each attack surface is paired with a legitimate case designed to verify that the framework does not reject normal traffic after processing an attack. These legitimate cases include benign tool descriptions, new non-colliding tool names, unchanged manifests, matching binary hashes, authorized role-tool invocations, valid API-key configuration, trusted origins, within-limit rate requests, benign tool arguments, clean memory context, valid external threat intelligence text, correct handoff sequences, benign server responses, large-but-permitted responses, and response plans with clear non-coercive justification.

A legitimate case is counted as a false block when it is rejected despite not violating the corresponding security policy. Therefore, the false-block metric should be interpreted only with respect to the legitimate cases included in the harness, not as a general false-positive rate for arbitrary SOC traffic.

3.5.4. Success Criteria

A subtest is considered successful when all the following conditions hold:

- the attack instance is blocked by the expected framework component;
- the attack does not reach the protected agent, MCP server, memory context, or analyst approval surface;
- the corresponding legitimate case is accepted;
- the result is stable across the configured number of trials.

These criteria reflect the objective of the evaluation: validating that each framework layer enforces its intended security property while preserving legitimate SOC pipeline behavior under the controlled laboratory threat model.

4. Results

This section reports the experimental results obtained under the evaluation methodology and test harness described in Sections 3.4 and 3.5.

4.1. Test Suite Results

Table 12 summarizes the results across all six suites. The Vectors column counts attack vectors exercised per test at a finer granularity than Table 2; some vectors are exercised across more than one test, so the column total is not expected to equal the 29 covered vectors. The Unique column reports the number of distinct attack instances per suite, which differs from the Vectors column because a single instance may exercise more than one vector and, conversely, a vector may be split across instances. Every one of the 25 unique attack instances was blocked in all 20 of its executions, yielding 600 correct block decisions in total, none bypassed the framework, and no legitimate call in the evaluated set was incorrectly rejected in any suite.

Table 12. Evaluation results summary. Unique: distinct attack instances exercised by the suite, each repeated over $N = 20$ executions. Blocked: individual block decisions, correct/total; several instances evaluate more than one payload or artefact per execution, so the block-decision count per suite exceeds its instance count. Passed: attack decisions that bypassed all controls. FB: legitimate calls incorrectly blocked.

Test	Component	Vectors	Unique	Blocked	Passed	FB
T00	Tool registration validator	4	4	100/100	0	0
T01	Layer 1: Access control	8	5	120/120	0	0
T02	Layer 2: Rate limiter	4	2	40/40	0	0
T03	Layer 3: Input validator	5	5	100/100	0	0
T04	Layer 4: Output validator	7	7	160/160	0	0
T05	Session controls	3	2	80/80	0	0
Total			**25**	**600/600**	**0**	**0**

4.2. Full-Pipeline Demonstration

Beyond the per-layer evaluation, a full-pipeline demonstration executes the complete triage, enrichment, and response sequence under two conditions, an unprotected baseline with direct access to both MCP servers and the secured condition routing every call through the full framework, against both a real alert retrieved live from the Wazuh deployment and a synthetic SSH brute-force alert. In the baseline condition all four tools from the malicious server are admitted into the agent namespace; in the secured condition the connection-phase registration validator admits only two: the tool exploiting a name collision with the legitimate `get_wazuh_latest_alert` tool is rejected by the namespace ownership check, and the tool embedding adversarial directives in its description field is removed by the regex

scan. The remaining two tools, whose payloads are embedded in the response body rather than in tool metadata or name, are admitted at connection time and addressed downstream: the indirect prompt injection payload by Layer 4 (output validator), and the confused deputy attempt by Layer 1 (access control) together with Layer 3. Decision-relevant fields of the response plan, including recommended actions, the escalation flag, and the human approval requirement, were identical between the unprotected and secured conditions for both alerts. Both runs also recorded two calls rejected under a schema-validation outcome for an enrichment-stage call passing an unsupported wildcard status value, a rejection independently confirmed by the live Wazuh deployment itself; this indicates that Layer 3 anticipates a systematic model-side parameter assumption rather than the framework introducing spurious failures, sparing the corresponding network round-trip and producing a structured, attributable audit entry in place of a generic tool-call failure.

4.3. Performance Overhead

Each control was benchmarked in isolation: the relevant validator was instantiated directly and invoked on a representative legitimate input that passes all its checks, with the call wrapped in a high-resolution timer (`time.perf_counter`). For each control, 200 warm-up iterations were discarded to exclude one-time costs such as regular-expression compilation and module import, after which $N = 10{,}000$ timed iterations were recorded; the table reports their mean and 95th percentile. Measuring the accepted (allow) path in isolation, rather than end-to-end wall-clock time, is deliberate, since end-to-end timing is dominated by network and model-inference latency and does not isolate the framework's own processing cost. Table 13 reports the results. With the exception of JSON Schema argument validation, every control completes in under 20 µs. Argument validation costs 546.6 µs in the evaluated implementation because the schema is recompiled on each invocation; precompiling the validator once at initialization, a one-line change, reduces this to 11.4 µs. Even without that optimization, the aggregate cost of the controls traversed by a typical tool call (Layers 1–4), summed across those controls, is below 0.6 ms, and below 30 µs with schema precompilation. This overhead is three to four orders of magnitude smaller than the latency of a single language-model invocation in the pipeline, which dominates end-to-end response time; the framework's cost is therefore negligible relative to the operation it protects, and does not materially affect the real-time behavior of the pipeline. These figures measure the framework's local processing cost only; end-to-end latency is governed by network round-trips and model inference, which the framework does not alter for legitimate traffic.

Table 13. Local processing cost of each framework control, measured in isolation from network and model-inference latency ($N = 10{,}000$ repetitions per control, single-threaded). The connection-phase controls run once per server at startup; the execution-phase controls run per tool call.

Phase	Control	Mean (µs)	p95 (µs)
Connection	Manifest hash (SHA-256)	4.7	5.0
	Name-collision check	0.3	0.4
	Full tool registration	7.9	8.6
Execution	L1: access control	0.2	0.2
	L2: rate limiter	0.9	0.9
	L3: argument validation (JSON Schema) [†]	546.6	592.5
	L3: memory-context scan	5.6	5.8
	L3: external-data scan	16.5	17.4
	L4: output validation	14.4	15.1

[†] The evaluated implementation recompiles the JSON Schema on each call; precompiling the validator once (a one-line change) reduces this control to 11.4 µs mean/10.8 µs p95.

4.4. Robustness to Obfuscation

The evaluation in Section 4.1 uses payloads in their literal form. Because the content-inspection controls in Layers 3 and 4 rely on fixed pattern matching, their robustness to obfuscated variants must be assessed separately. To do so, four injection payloads that the Layer 3 scan blocks in their clear form were subjected to four basic obfuscation techniques: base64 encoding, homoglyph substitution (Latin characters replaced by Cyrillic look-alikes), translation into a language not covered by the pattern sets (Spanish), and manual paraphrase preserving intent. Table 14 reports how many of the four payloads remain blocked under each technique. Every technique evaded the Layer 3 controls completely, a detection gap of 100% relative to clear-form detection; the Layer 4 scan detects two of the four payloads in clear form, and both are likewise evaded by all four techniques. This is a structural consequence of literal, language-specific matching rather than a deficiency of the specific patterns chosen, and it delimits the scope of the blocking result reported above: the guarantee holds for the non-adaptive, in-clear payloads of the evaluated threat model and does not extend to obfuscated or mutated variants. The implications of this gap, and the semantic detection it motivates, are discussed in Section 5.

Table 14. Detection of four injection payloads under obfuscation **by the Layer 3 input scan**. Each payload is one that Layer 3 blocks in its clear form; the column reports how many remain blocked after each technique.

Obfuscation Technique	Description	Blocked (of 4)
None (clear)	Literal payload as derived from the threat model	4
Base64	Payload base64-encoded within a carrier string	0
Homoglyph	Latin characters replaced by Cyrillic look-alikes	0
Multilingual	Semantically equivalent payload in Spanish	0
Paraphrase	Reworded to preserve intent without literal triggers	0

4.5. False Positives on Attack-like Legitimate Traffic

The false-block metric in Section 4.1 is measured over legitimate cases that do not resemble attack payloads. Because a SOC routinely handles text that legitimately contains attack-like substrings (command-line logs, file paths, CVE descriptions, and tickets that quote adversarial instructions verbatim), the behavior of the Layer 3 injection scan on such inputs was assessed separately. Table 15 reports eight benign inputs of this kind passed through the argument-injection scan in isolation. Seven of the eight were blocked as false positives: the shell-metacharacter pattern flags ordinary command-line logs, file paths, and environment-variable syntax, and the instruction-override patterns flag tickets or advisories that merely quote or describe an attack. The single input that passed, a CVE description mentioning `eval()`, is instead blocked when routed through the external-data channel, whose extended pattern set includes that construct, illustrating that the outcome depends on the channel rather than on the content being genuinely malicious. This is the same structural property observed in Section 4.4, seen from the opposite side: literal pattern matching produces false negatives on obfuscated attacks and false positives on benign traffic that shares surface features with attacks. The "no legitimate call in the evaluated set was incorrectly rejected" result therefore holds for the benign legitimate cases of the evaluated harness and does not extend to attack-like legitimate inputs, whose false-positive rate reinforces semantic detection as the primary avenue of future work (Section 5).

Table 15. Behavior of the Layer 3 injection scan on legitimate SOC inputs that contain attack-like substrings. Each case is benign traffic; "blocked" indicates a false positive of the argument-injection scan evaluated in isolation.

Legitimate Input (Benign SOC Traffic)	Scan Result
Command-line log with shell metacharacters (`&`, `|`)	Blocked
Firewall/process log with pipe and semicolon	Blocked
Windows file path with backslashes	Blocked
CVE description mentioning `eval()`	Passed
Analyst ticket quoting an attacker instruction verbatim	Blocked
Threat-intel note about a prompt-extraction campaign	Blocked
Log line with environment-variable syntax (`$PATH`)	Blocked
Detection rule text referencing PowerShell and an override phrase	Blocked

5. Discussion

The results reported in Section 4 indicate that systematic, middleware-level controls can address the documented MCP-specific attack surface without modifying the underlying agents or servers. Across 600 executions (25 unique attack instances repeated 20 times each) spanning six independently isolated test suites, every attack instance was blocked, none bypassed the framework, and no legitimate call in the evaluated set was incorrectly rejected, providing experimental support for the design criteria derived from the literature in Section 2. The full-pipeline demonstration further indicates that these controls do not alter the operational output of the system in the absence of an attack, addressing a concern raised implicitly by prior work on tool-input and tool-output firewalls [30], namely that lightweight, model-agnostic defenses risk degrading usability when applied indiscriminately to all traffic.

These findings extend the architectural literature reviewed in Section 2 in two respects. First, where AgenticCyOps [15] formalizes capability scoping, verified execution, and memory integrity as defensive principles without an implemented, experimentally evaluated middleware layer, this work operationalizes equivalent principles as concrete, testable controls and reports their behavior under 600 controlled executions rather than at the level of architectural argument. Second, where Guo et al. [16] characterize tool poisoning as a pervasive and largely unaddressed vulnerability class, the connection-phase tool registration validator proposed here demonstrates that pre-admission inspection of tool metadata is both implementable and effective against the description-poisoning and name-collision variants exercised in T00, without requiring any modification to the underlying language model.

Despite these results, several limitations constrain the generalizability of the findings. The evaluation was conducted against a single, purpose-built malicious server whose payloads represent documented vectors from the literature but do not capture the full variability of adaptive, real-world adversarial inputs; an attacker who observes framework behavior and iteratively refines payloads to evade the regex-based pattern sets used in Layers 3 and 4 was not modeled. Accordingly, the failure-opacity criterion (Section 3.2) is claimed only as an attack-surface-reduction measure that withholds the diagnostic signal an adaptive attacker would exploit, and is verified structurally rather than against an iterative probing campaign; establishing its empirical effect against an adaptive adversary requires the adaptive evaluation identified below as future work. This is a structural limitation of static pattern matching rather than of the specific patterns selected. As motivated in Section 3.2, deterministic matching was adopted deliberately to preserve reproducibility, auditability, and model independence; replacing it with semantic, classifier- or LLM-based detection is therefore the most direct extension of this work rather than a baseline it should

have adopted, at the cost of additional latency and computational overhead, particularly in the output validator, which already carries the largest response-scanning burden in the current evaluation.

The consequences of relying on lexical rather than semantic detection are concrete and worth enumerating, since they delimit the specific inputs against which Layers 3 and 4 offer no guarantee. Four scenarios are representative. First, a tool description or retrieved payload may be semantically malicious yet lexically benign: an instruction rephrased to induce a harmful action without using any of the imperative or coercive constructions the pattern sets target (for example, framing an exfiltration step as a routine diagnostic recommendation) would not match and would reach the agent. Second, an equivalent payload expressed in a language other than those covered by the pattern sets would evade scanning entirely, since the patterns are language-specific. Third, encoding or obfuscation, such as base64, homoglyph substitution, or fragmentation of a trigger phrase across fields, defeats literal matching because the surface string no longer contains the pattern. Fourth, a paraphrase of a known injection that preserves its intent while altering its wording falls outside a fixed pattern by construction. In each case the failure mode is a false negative confined to the content-inspection controls; the structural controls (role-based access control, rate limiting, schema validation, session isolation, and the connection-phase integrity checks) are unaffected, so an attack that evades Layers 3 and 4 is still bounded by least-privilege enforcement and cannot, for instance, invoke a tool outside the compromised agent's role. These scenarios define the residual risk that motivates semantic, classifier- or LLM-based detection as the primary avenue of future work. This risk is not merely hypothetical: the obfuscation experiment reported in Section 4.4 confirms it quantitatively, with all four techniques evading the lexical controls entirely. The converse failure mode is equally concrete: Section 4.5 shows that the same lexical controls block legitimate SOC traffic containing attack-like substrings, with seven of eight representative benign inputs flagged as false positives. Lexical matching is therefore imprecise in both directions, which is the core argument for semantic detection.

Six of the 35 middleware-addressable vectors identified in the threat model are not covered, as detailed in Section 3.2.8; three are outside the scope of a middleware-level framework by design; two, consensus manipulation and covert coordination, are inapplicable to the sequential pipeline topology evaluated here but would become relevant in non-sequential, decentralized, or hierarchical multi-agent configurations; and replay is not addressed at the middleware layer, since effective replay protection requires request-freshness verification at the receiving server rather than at the interception point. Extending the framework to such topologies, in particular generalizing the handoff sequence definition from a linear map to a transition graph as outlined in Section 3.3, is required before these vectors can be addressed. The DNS rebinding protection described in Section 3.2 is conditional on an explicit allowed origin being configured; in the containerized deployment evaluated here the check was inactive, and the property held through the loopback port binding and network segmentation described in Section 3.2, not through the application-layer check, and deployments without equivalent network-level isolation would need to enable it explicitly.

Finally, the architecture was evaluated against a single LLM, GPT-4o mini, and the file-based memory store, while adequate for transparency and reproducibility in a laboratory setting, is not suitable for production deployments requiring concurrent access guarantees at scale. Because all framework controls operate at the tool invocation and response layer rather than at the inference layer, the security properties reported here are expected to generalize across models; however, the extent to which model-level differences in

instruction-following behavior affect the residual risk reaching agents after framework controls have been applied remains an open empirical question.

Taken together, these results hold under a specific and explicitly bounded set of conditions, and it is worth stating both sides of that boundary directly. The framework's blocking guarantee is established for non-adaptive, in-clear payloads representative of the documented threat model, delivered against a sequential triage-enrichment-response pipeline, under the network-segmented containerized deployment evaluated here, and with security decisions that are by construction independent of the underlying language model. Within those conditions, every evaluated attack instance was blocked and no legitimate case in the evaluated set was rejected. The result is correspondingly not established, and should not be read as holding, for adaptive adversaries that obfuscate or iteratively refine payloads to evade fixed pattern sets; for non-sequential, decentralized, or hierarchical multi-agent topologies, against which the consensus-manipulation and covert-coordination vectors become relevant; for deployments lacking the network-level isolation on which the DNS-rebinding property depends by default; or, quantitatively, beyond the single language model and the file-based memory backend used in this laboratory setting. The contribution is therefore best understood as evidence that systematic middleware controls close the documented, non-adaptive MCP attack surface under realistic but controlled conditions, and as a foundation on which the adaptive, topological, and cross-model evaluations identified above remain necessary future work.

6. Conclusions

This paper proposed a security framework for MCP-based multi-agent systems in SOC environments, implemented as a middleware layer comprising a connection-phase tool registration validator and five execution layers covering access control, rate limiting, input validation, output validation, and audit logging. The framework was applied to a three-agent triage-enrichment-response pipeline with persistent shared memory, connected to a real Wazuh SIEM deployment through a custom MCP server, and evaluated against a purpose-built, specification-compliant malicious MCP server simulating tool shadowing, tool description poisoning, indirect prompt injection, and confused deputy attacks under controlled, reproducible conditions.

The architecture and framework jointly address 29 of 35 middleware-addressable attack vectors identified from the agentic AI and MCP security literature, selected according to explicit architectural-relevance, empirical-grounding, and SOC-applicability criteria. Across six independently isolated test suites totaling 25 unique attack instances and 600 executions, every attack instance in the evaluated threat model was blocked, no attack bypassed the framework, and no legitimate call in the evaluated set was incorrectly rejected. A separate probe showed that obfuscated variants of these payloads (encoding, homoglyphs, paraphrase, or non-English injection) evade the lexical content-inspection controls, delimiting the scope of this result (Section 4.4). The same lexical mechanism produces false positives on legitimate traffic that contains attack-like substrings, as reported in Section 4.5. A full-pipeline demonstration further confirmed that the framework leaves the operational output of the system unchanged in the absence of an attack, against both a real Wazuh alert and a synthetic scenario.

These results support the conclusion that systematic, middleware-level controls, designed from an explicit, literature-derived threat model and validated experimentally rather than only architecturally, can secure MCP-based agentic SOC deployments without requiring modification to the underlying agents or MCP servers. As discussed in Section 5, the principal directions for future work concern replacing static pattern matching with semantic detection mechanisms, particularly in the output validation layer; extending the framework

and the underlying architecture to non-sequential multi-agent topologies; evaluating the framework against a broader range of underlying language models; transitioning the laboratory implementation toward a production-grade deployment, including a scalable memory backend and integration of the audit log with existing SIEM monitoring infrastructure for real-time anomaly detection; strengthening the audit log's tamper-evidence beyond the framework's own execution context, for instance through hash chaining between consecutive entries or write-once storage; and periodically re-applying the three-criterion selection methodology of Section 3.2.8 to incorporate emerging attack vectors as the referenced taxonomies, including the OWASP MCP Top 10, reach a stable release.

Author Contributions: Conceptualization, R.T.d.P.S. and X.L.-N.; methodology, R.T.d.P.S., X.L.-N. and C.S.-Z.; software, R.T.d.P.S.; validation, R.T.d.P.S. and X.L.-N.; formal analysis, R.T.d.P.S.; investigation, R.T.d.P.S. and X.L.-N.; resources, V.A.V. and A.I.M.L.; data curation, R.T.d.P.S.; writing—original draft preparation, R.T.d.P.S.; writing—review and editing, R.T.d.P.S., X.L.-N., C.S.-Z., V.A.V. and A.I.M.L.; visualization, R.T.d.P.S.; supervision, X.L.-N. and C.S.-Z.; project administration, V.A.V. and A.I.M.L.; funding acquisition, A.I.M.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the R&D activity program with reference TEC-2024/COM-504 and acronym RAMONES-CM, granted by the Comunidad de Madrid through the Directorate General for Research and Technological Innovation via Order 5696/2024.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study, including the source code of the multi-agent pipeline, the security framework, the malicious MCP server, and the test harness used to generate the results reported in Section 4, are openly available in the project repository at https://github.com/UPM-RSTI/MCP_Sec_Framework (accessed on 15 July 2026) [34]. The audit logs and evaluation results reported in this study can be regenerated by executing the provided test suite against the laboratory environment described in Section 3.3.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

SOC	Security Operations Center
MCP	Model Context Protocol
LLM	Large Language Model
AI	Artificial Intelligence
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation, and Response
RBAC	Role-Based Access Control
HITL	Human-in-the-Loop
JSON	JavaScript Object Notation
JSON-RPC	JSON Remote Procedure Call
SSE	Server-Sent Events
API	Application Programming Interface
REST	Representational State Transfer
UUID	Universally Unique Identifier
SHA	Secure Hash Algorithm
DNS	Domain Name System
CSRF	Cross-Site Request Forgery
CORS	Cross-Origin Resource Sharing

RAG	Retrieval-Augmented Generation
DoS	Denial of Service

References

1. Khayat, M.; Barka, E.; Serhani, M.A.; Sallabi, F.; Shuaib, K.; Khater, H.M. Empowering Security Operation Center With Artificial Intelligence and Machine Learning—A Systematic Literature Review. *IEEE Access* **2025**, *13*, 19162–19197. [CrossRef]
2. Tariq, S.; Baruwal Chhetri, M.; Nepal, S.; Paris, C. Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities. *ACM Comput. Surv.* **2025**, *57*, 224. [CrossRef]
3. Baruwal Chhetri, M.; Tariq, S.; Singh, R.; Jalalvand, F.; Paris, C.; Nepal, S. Towards Human-AI Teaming to Mitigate Alert Fatigue in Security Operations Centres. *ACM Trans. Internet Technol.* **2024**, *24*, 12. [CrossRef]
4. Alshammari, T.; Al-Balawi, T. Systematic Literature Review: Challenges and Issues in the Adoption of SOAR Technology in Cybersecurity. *Int. Conf. Cyber Warf. Secur.* **2026**, *21*, 9–17. [CrossRef]
5. Palo Alto Networks. Security Operations in 2025 and Beyond. 2024. Available online: https://www.paloaltonetworks.com/blog/2024/12/security-operations-in-2025-and-beyond/ (accessed on 15 July 2026).
6. Patil, A.; Jadon, A. Advancing Reasoning in Large Language Models: Promising Methods and Approaches. *arXiv* **2025**, arXiv:2502.03671v2. [CrossRef]
7. Wang, Y.; Pan, Y.; Su, Z.; Deng, Y.; Zhao, Q.; Du, L.; Luan, T.H.; Kang, J.; Niyato, D. Large Model-Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends. *IEEE Commun. Surv. Tutor.* **2026**, *28*, 1906–1949. [CrossRef]
8. Renney, H.; Nethercott, M.N.; Hayes, P.; Renney, N. LLM-Enabled Multi-Agent Systems: Empirical Evaluation and Insights into Emerging Design Patterns & Paradigms. *arXiv* **2026**, arXiv:2601.03328. [CrossRef]
9. Krishnan, N. Advancing Multi-Agent Systems Through Model Context Protocol: Architecture, Implementation, and Applications. *arXiv* **2025**, arXiv:2504.21030. [CrossRef]
10. Anthropic. Introducing the Model Context Protocol. 2024. Available online: https://www.anthropic.com/news/model-context-protocol (accessed on 15 July 2026).
11. Hou, X.; Zhao, Y.; Wang, S.; Wang, H. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. *ACM Trans. Softw. Eng. Methodol.* 2026, *accepted*. [CrossRef]
12. Narajala, V.S.; Habler, I. Enterprise-Grade Security for the Model Context Protocol (MCP): Frameworks and Mitigation Strategies. *arXiv* **2025**, arXiv:2504.08623. [CrossRef]
13. Pento. A Year of MCP: From Internal Experiment to Industry Standard. 2025. Available online: https://www.pento.ai/blog/a-year-of-mcp-2025-review (accessed on 15 July 2026).
14. Gupta, D. The Complete Guide to Model Context Protocol (MCP): Enterprise Adoption, Market Trends, and Implementation Strategies. 2025. Available online: https://guptadeepak.com/the-complete-guide-to-model-context-protocol-mcp-enterprise-adoption-market-trends-and-implementation-strategies/ (accessed on 15 July 2026).
15. Mitra, S.; Patel, R.; Mittal, S.; Rahman, M.R.; Rahimi, S. AgenticCyOps: Securing Multi-Agentic AI Integration in Enterprise Cyber Operations. *arXiv* **2026**, arXiv:2603.09134. [CrossRef]
16. Guo, Y.; Liu, P.; Ma, W.; Deng, Z.; Zhu, X.; Di, P.; Xiao, X.; Wen, S. MCPXKIT: The Unified Toolkit for Analyzing Model Context Protocol Security. *IEEE Trans. Dependable Secur. Comput.* **2026**, 1–16. [CrossRef]
17. Gulyamov, S.; Gulyamov, S.; Rodionov, A.; Khursanov, R.; Mekhmonov, K.; Babaev, D.; Rakhimjonov, A. Prompt Injection Attacks in Large Language Models and AI Agent Systems: A Comprehensive Review of Vulnerabilities, Attack Vectors, and Defense Mechanisms. *Information* **2026**, *17*, 54. [CrossRef]
18. Chang, H.; Bao, E.; Luo, X.; Yu, T. Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems. *arXiv* **2026**, arXiv:2601.07072. [CrossRef]
19. Lupinacci, M.; Pironti, F.A.; Blefari, F.; Romeo, F.; Arena, L.; Furfaro, A. The Dark Side of LLMs: Agent-based Attack Vectors for System-level Compromise. *arXiv* **2026**, arXiv:2507.06850. [CrossRef]
20. OWASP GenAI Security Project. Agentic AI-Threats and Mitigations. 2025. Available online: https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/ (accessed on 15 July 2026).
21. OWASP Foundation. OWASP Top 10 for Large Language Model Applications. 2025. Available online: https://owasp.org/www-project-top-10-for-large-language-model-applications/ (accessed on 15 July 2026).
22. OWASP GenAI Security Project. OWASP Top 10 for Agentic Applications 2026. 2025. Available online: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/ (accessed on 15 July 2026).
23. OWASP Foundation. OWASP MCP Top 10. Incubator Project, Phase 3 Beta. Version 0.0.0. 2025. Available online: https://owasp.org/www-project-mcp-top-10/ (accessed on 15 July 2026).
24. MITRE Corporation. MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems. v5.1.0. 2015. Available online: https://atlas.mitre.org (accessed on 15 July 2026).

25. Narajala, V.S.; Narayan, O. Securing Agentic AI: A Comprehensive Threat Model and Mitigation Framework for Generative AI Agents. *arXiv* **2025**, arXiv:2504.19956. [CrossRef]
26. Shen, Y.T.; Toyoda, K.; Leung, A. MCP-38: A Comprehensive Threat Taxonomy for Model Context Protocol Systems (v1.0). *arXiv* **2026**, arXiv:2603.18063. [CrossRef]
27. Raza, S.; Sapkota, R.; Karkee, M.; Emmanouilidis, C. TRiSM for Agentic AI: A review of Trust, Risk, and Security Management in LLM-based Agentic Multi-Agent Systems. *AI Open* **2026**, *7*, 71–95. [CrossRef]
28. Chhabra, A.; Datta, S.; Nahin, S.K.; Mohapatra, P. Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges. *IEEE Access* **2026**, *14*, 49455–49482. [CrossRef]
29. Kim, J.; Liu, X.; Wang, Z.; Qiu, S.; Li, B.; Guo, W.; Song, D. The Attack and Defense Landscape of Agentic AI: A Comprehensive Survey. *arXiv* **2026**, arXiv:2603.11088. [CrossRef]
30. Bhagwatkar, R.; Kasa, K.; Puri, A.; Huang, G.; Rish, I.; Taylor, G.W.; Dvijotham, K.D.; Lacoste, A. Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks? *arXiv* **2025**, arXiv:2510.05244. [CrossRef]
31. Hossain, S.M.A.; Shayoni, R.K.; Ameen, M.R.; Islam, A.; Mridha, M.F.; Shin, J. A Multi-Agent LLM Defense Pipeline Against Prompt Injection Attacks. *arXiv* **2025**, arXiv:2509.14285. [CrossRef]
32. Brigandi, G. mcp-server-wazuh: Wazuh SIEM MCP Server. 2024. Available online: https://github.com/gbrigandi/mcp-server-wazuh (accessed on 29 May 2026).
33. XenonStack. Human-in-the-Loop in SOC Automation. 2024. Available online: https://www.xenonstack.com/blog/human-loop-soc-automation (accessed on 15 July 2026).
34. Tavares de Pina Simões, R. Security Framework for MCP-Based Multi-Agent Systems in SOC Environments. Code Repository for Master's Thesis Implementation, Universidad Politécnica de Madrid. 2026. Available online: https://github.com/UPM-RSTI/MCP_Sec_Framework (accessed on 27 June 2026).
35. OWASP GenAI Security Project. A Practical Guide for Secure MCP Server Development. 2026. Available online: https://genai.owasp.org/resource/a-practical-guide-for-secure-mcp-server-development/ (accessed on 15 July 2026).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.